# Estimación de latencia en microservicios mediante interpolación polinomial de Newton

*Latencia esperada de una API REST cuando opera a 500 req/s*

Estudiante: **Fernando Istaña Flores** Escuela: **Ingeniería de Sistemas**

## 1. Contexto y datos

En una arquitectura de microservicios desplegada en un entorno Cloud, el equipo de SRE (Site Reliability Engineering) evalúa la degradación de rendimiento de una API REST crítica a medida que aumenta la carga de peticiones concurrentes por segundo (x, en cientos de req/s). La variable de respuesta es la latencia media (y, en milisegundos). Se obtuvieron mediciones en cuatro puntos de carga específicos.

| Punto | Carga concurrente x (× 100 req/s) | Latencia medida y = f(x) (ms) |
|---|---|---|
| x0 | 1 | 45 |
| x1 | 2 | 65 |
| x2 | 4 | 110 |
| x3 | 7 | 220 |

Con cuatro nodos corresponde un polinomio interpolador de grado n = 3. El objetivo es estimar la latencia cuando el sistema opera a **500 req/s (x = 5)**, un nivel de carga que no se midió. Los nodos están separados por 1, 2 y 3 unidades: no son equiespaciados, por lo que se usan diferencias divididas y no la fórmula de Newton-Gregory.

## 2. Construcción manual — Tabla de diferencias divididas

El polinomio interpolante de Newton tiene la forma

```
P₃(x) = f[x0] + f[x0,x1](x − x0) + f[x0,x1,x2](x − x0)(x − x1)
        + f[x0,x1,x2,x3](x − x0)(x − x1)(x − x2)
```

Sus coeficientes son las diferencias divididas, que se obtienen columna por columna a partir de los valores de la función:

```
f[x_i, …, x_{i+j}] = ( f[x_{i+1}, …, x_{i+j}] − f[x_i, …, x_{i+j−1}] ) / ( x_{i+j} − x_i )
```

### Paso 1 — Primeras diferencias

```
f[1, 2] = (65 − 45)   / (2 − 1) = 20
f[2, 4] = (110 − 65)  / (4 − 2) = 45/2  = 22.5
f[4, 7] = (220 − 110) / (7 − 4) = 110/3 = 36.6667
```

### Paso 2 — Segundas diferencias

```
f[1, 2, 4] = (22.5 − 20)      / (4 − 1) = 5/6  = 0.8333
f[2, 4, 7] = (36.6667 − 22.5) / (7 − 2) = 17/6 = 2.8333
```

### Paso 3 — Tercera diferencia

```
f[1, 2, 4, 7] = (2.8333 − 0.8333) / (7 − 1) = 1/3 = 0.3333
```

*En la segunda y la tercera columna el denominador es la resta de los x de los extremos del grupo (4 − 1, 7 − 2 y 7 − 1), no de dos nodos vecinos.*

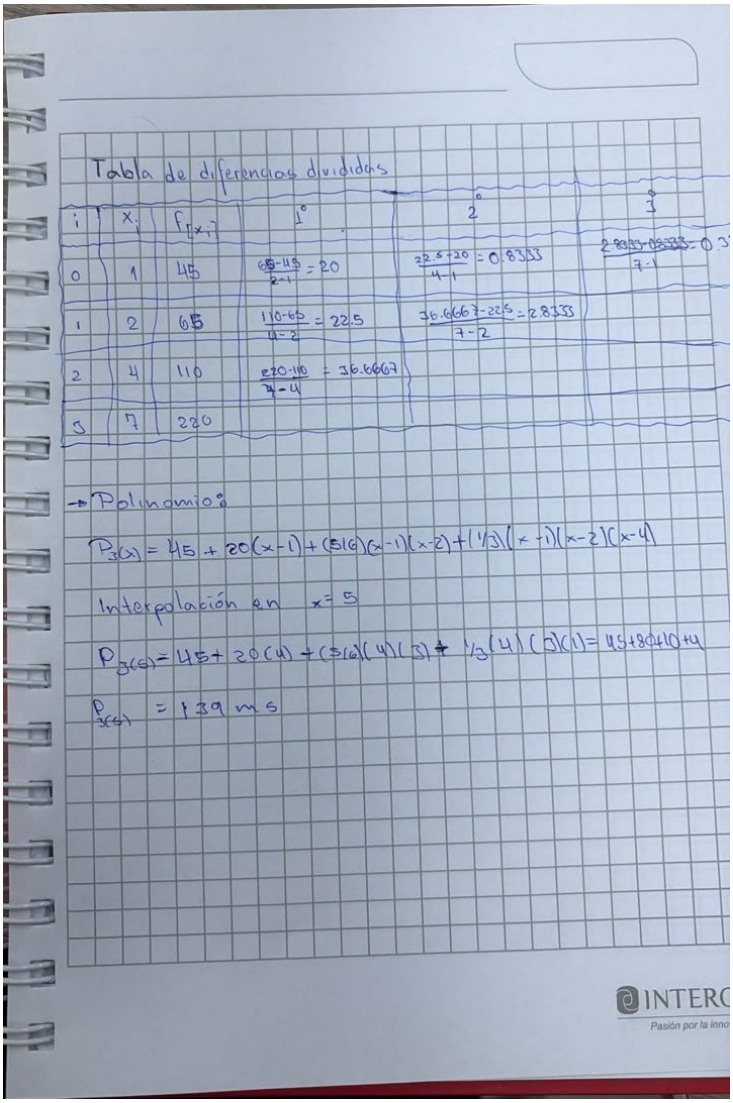
### Tabla completa

| i | x_i | f[x_i] | 1.ª diferencia | 2.ª diferencia | 3.ª diferencia |
|---|---|---|---|---|---|
| 0 | 1 | **45** | **20** | **5/6 = 0.8333** | **1/3 = 0.3333** |
| 1 | 2 | 65 | 45/2 = 22.5 | 17/6 = 2.8333 | |
| 2 | 4 | 110 | 110/3 = 36.6667 | | |
| 3 | 7 | 220 | | | |

Los coeficientes del polinomio son los valores de la primera fila: **45, 20, 5/6 y 1/3**.

## 3. Desarrollo manual

A continuación se adjunta el desarrollo realizado a mano, del que proceden los valores tabulados en la sección anterior.




*Hoja 1: tabla de diferencias divididas con el cálculo de cada casilla, polinomio en forma de Newton e interpolación en x = 5.*

## 4. Polinomio interpolador de grado 3

Con la primera fila de la tabla, la forma de Newton es:

$$P_3(x) = 45 + 20(x - 1) + (5/6)(x - 1)(x - 2) + (1/3)(x - 1)(x - 2)(x - 4)$$

### Paso 1 — Desarrollar cada término

| k | Término | Desarrollado |
|---|---|---|
| 0 | 45 | 45 |
| 1 | 20(x − 1) | 20x − 20 |
| 2 | (5/6)(x − 1)(x − 2) | (5/6)x² − (5/2)x + 5/3 |
| 3 | (1/3)(x − 1)(x − 2)(x − 4) | (1/3)x³ − (7/3)x² + (14/3)x − 8/3 |

### Paso 2 — Sumar los coeficientes de igual grado

```
x³ :  1/3                                          = 1/3
x² :  5/6 − 7/3              = (5 − 14) / 6        = −3/2
x¹ :  20 − 5/2 + 14/3        = (120 − 15 + 28) / 6 = 133/6
x⁰ :  45 − 20 + 5/3 − 8/3    = 25 − 1              = 24
```

$$P_3(x) = (1/3)\cdot x^3 - (3/2)\cdot x^2 + (133/6)\cdot x + 24 \quad \approx \quad 0.333333x^3 - 1.5x^2 + 22.166667x + 24$$

## 5. Interpolación en x = 5 (500 req/s)

Se sustituye x = 5 en la forma de Newton. Cada término reutiliza el producto del anterior, de modo que el reemplazo numérico es corto:

| k | Coeficiente | Producto (x − x_j) | Término | Suma acumulada |
|---|---|---|---|---|
| 0 | 45 | 1 | 45 | 45 |
| 1 | 20 | (5 − 1) = 4 | 80 | 125 |
| 2 | 5/6 | (5 − 1)(5 − 2) = 12 | 10 | 135 |
| 3 | 1/3 | (5 − 1)(5 − 2)(5 − 4) = 12 | 4 | **139** |

**$P_3(5) = 45 + 80 + 10 + 4 = 139$ ms.** La latencia media esperada cuando la API opera a 500 req/s es de aproximadamente 139 milisegundos.

*Verificación por la forma estándar: (1/3)(125) − (3/2)(25) + (133/6)(5) + 24 = 41.6667 − 37.5 + 110.8333 + 24 = 139 ms. Ambos caminos coinciden.*

## 6. Validación teórica de la interpolación

El polinomio debe pasar exactamente por los cuatro puntos experimentales. Al evaluar en un nodo x_k se anulan todos los términos que contienen el factor (x − x_k), y la suma de los que quedan reproduce el dato:

| x_i | Sustitución en la forma de Newton | $P_3(x_i)$ | Dato y_i |
|---|---|---|---|
| 1 | 45 | 45 | 45 |
| 2 | 45 + 20(1) | 65 | 65 |
| 4 | 45 + 20(3) + (5/6)(3)(2) | 110 | 110 |
| 7 | 45 + 20(6) + (5/6)(6)(5) + (1/3)(6)(5)(3) | 220 | 220 |

Los cuatro valores coinciden con las mediciones, lo que confirma que la tabla y los coeficientes están bien calculados. Como el polinomio interpolador de un conjunto de puntos es único, este mismo $P_3(x)$ se obtendría con Lagrange; la ventaja de Newton es que agregar una medición más solo añade un término, sin rehacer los anteriores.

## 7. Aplicativo solucionador

El aplicativo automatiza el proceso completo. Recibe los vectores de cargas X y de latencias Y, construye la matriz triangular con la tabla de diferencias divididas mediante la función diferencias_divididas(X, Y), evalúa el polinomio con la regla anidada de Newton y se valida con los datos del caso de estudio calculando $P_3(5)$. La versión web permite además explorar la curva de latencia con un deslizador y comparar el grado del polinomio en tiempo real.

### Comparación de grados

Con grado k solo se usan los primeros k + 1 puntos, y cada polinomio es el anterior más un término:

| Grado | Nodos | Polinomio | P_k(5) (ms) |
|---|---|---|---|
| 1 | 1, 2 | $P_1(x) = 45 + 20(x - 1)$ | 125 |
| 2 | 1, 2, 4 | $P_2(x) = P_1(x) + (5/6)(x - 1)(x - 2)$ | 135 |
| 3 | 1, 2, 4, 7 | $P_3(x) = P_2(x) + (1/3)(x - 1)(x - 2)(x - 4)$ | **139** |

*La recta subestima la latencia porque ignora la curvatura; las correcciones al subir de grado son cada vez menores (+10 y +4 ms), señal de que la estimación se estabiliza.*

### Motor de cálculo

```
"""
Análisis Numérico · Unidad 2 · Sesión 9
Interpolación polinomial de Newton — latencia media de una API REST según la carga concurrente.

Datos de las pruebas del equipo de SRE:
    x (x 100 req/s) :   1     2     4     7
    y (ms)          :  45    65   110   220

Objetivo: estimar la latencia cuando el sistema opera a 500 req/s (x = 5).

Fernando Istaña Flores · Universidad Peruana Unión
"""

from fractions import Fraction


# ---------------------------------------------------------------------------
# Construcción de la matriz: tabla completa de diferencias divididas
# ---------------------------------------------------------------------------
def diferencias_divididas(X, Y):
    """
    Devuelve la matriz triangular n x n con la tabla completa.

        T[i][j] = f[x_i, x_{i+1}, ..., x_{i+j}]
        T[i][j] = (T[i+1][j-1] - T[i][j-1]) / (x_{i+j} - x_i)

    La columna 0 son los valores de f y la PRIMERA FILA son los coeficientes
    del polinomio de Newton. Debajo de la antidiagonal la matriz queda en cero.
    """
    n = len(X)
    if len(Y) != n:
        raise ValueError("X e Y deben tener el mismo tamaño.")
    if len(set(X)) != n:
        raise ValueError("Hay nodos repetidos: se dividiría entre cero.")
    T = [[0] * n for _ in range(n)]
    for i in range(n):
        T[i][0] = Y[i]
    for j in range(1, n):                # columna = orden de la diferencia
```

```
        for i in range(n - j):           # fila
            T[i][j] = (T[i + 1][j - 1] - T[i][j - 1]) / (X[i + j] - X[i])
    return T


# ---------------------------------------------------------------------------
# Evaluación eficiente: multiplicación anidada (el Horner de la forma de Newton)
# ---------------------------------------------------------------------------
def evaluar_newton(coefs, X, x):
    """
    P(x) = c0 + (x - x0)·(c1 + (x - x1)·(c2 + ... + (x - x_{n-1})·cn))

    Se recorre de adentro hacia afuera: n multiplicaciones y n sumas.
    Evaluar término por término costaría del orden de n^2 / 2 multiplicaciones.
    """
    p = coefs[-1]
    for k in range(len(coefs) - 2, -1, -1):
        p = coefs[k] + (x - X[k]) * p
    return p


# ---------------------------------------------------------------------------
# Forma estándar a_n x^n + ... + a_1 x + a_0
# ---------------------------------------------------------------------------
def forma_estandar(coefs, X):
    """Desarrolla la forma de Newton y devuelve [a_n, ..., a_1, a_0]."""
    p = [coefs[-1]]
    for k in range(len(coefs) - 2, -1, -1):
        nuevo = p + [0]                  # p(x) · x
        for i, a in enumerate(p):
            nuevo[i + 1] -= X[k] * a     # - x_k · p(x)
        nuevo[-1] += coefs[k]            # + c_k
        p = nuevo
    return p


def newton_texto(coefs, X):
    partes = []
    for k, c in enumerate(coefs):
        if c == 0:
            continue
        factores = "".join("(x %s %s)" % ("-" if X[j] >= 0 else "+", abs(X[j])) for j in range(k))
        val = "%s" % abs(c)
        if "/" in val and factores:
            val = "(" + val + ")"
        signo = " + " if c > 0 and partes else (" - " if c < 0 and partes else ("-" if c < 0 else ""))
        partes.append(signo + val + factores)
    return "".join(partes)


def polinomio_texto(coef):
    grados = len(coef) - 1
    partes = []
    for j, c in enumerate(coef):
        g = grados - j
        if c == 0:
            continue
        signo = " + " if c > 0 and partes else (" - " if c < 0 and partes else ("-" if c < 0 else ""))
        val = abs(c)
        term = "%s" % val if g == 0 else ("%s·x" % val if g == 1 else "%s·x^%d" % (val, g))
        partes.append(signo + term)
    return "".join(partes)
```

```
if __name__ == "__main__":
    # ---------------------------------------------------------------------
    # Entrada de datos: dos arreglos del mismo tamaño
    # ---------------------------------------------------------------------
    X = [1, 2, 4, 7]            # vector de cargas (x 100 req/s)
    Y = [45, 65, 110, 220]      # vector de latencias (ms)
    x_eval = 5                  # 500 req/s
    n = len(X) - 1

    print("=" * 74)
    print("DATOS EXPERIMENTALES")
    print("=" * 74)
    print("   i   x_i (x100 req/s)     y_i (ms)")
    for i, (a, b) in enumerate(zip(X, Y)):
        print("  %2d   %12.2f   %12.2f" % (i, a, b))

    # con fracciones la tabla sale exacta (5/6 en lugar de 0.8333...)
    Xf = [Fraction(v) for v in X]
    Yf = [Fraction(v) for v in Y]
    T = diferencias_divididas(Xf, Yf)
    coefs = T[0]

    print()
    print("=" * 74)
    print("TABLA DE DIFERENCIAS DIVIDIDAS (matriz triangular)")
    print("=" * 74)
    cab = ["i", "x_i", "f[x_i]"] + ["%d.a dif." % j for j in range(1, len(X))]
    print("  " + "".join("%11s" % c for c in cab))
    for i in range(len(X)):
        fila = [str(i), str(Xf[i])] + [str(T[i][j]) for j in range(len(X) - i)]
        print("  " + "".join("%11s" % c for c in fila))
    print()
    print("  En decimales:")
    Td = diferencias_divididas(X, [float(v) for v in Y])
    for i in range(len(X)):
        fila = [str(i), "%g" % X[i]] + ["%.4f" % Td[i][j] for j in range(len(X) - i)]
        print("  " + "".join("%11s" % c for c in fila))
    print()
    print("  Coeficientes (primera fila): " + ", ".join(str(c) for c in coefs))

    print()
    print("=" * 74)
    print("POLINOMIO INTERPOLADOR")
    print("=" * 74)
    est = forma_estandar(coefs, Xf)
    print("  Forma de Newton : P%d(x) = %s" % (n, newton_texto(coefs, Xf)))
    print("  Forma estándar  : P%d(x) = %s" % (n, polinomio_texto(est)))
    print("  coeficientes decimales: " + ", ".join("%+.6f" % float(c) for c in est))

    print()
    print("=" * 74)
    print("EVALUACIÓN ANIDADA EN x = %g" % x_eval)
    print("=" * 74)
    b = coefs[-1]
    print("  b%d = c%d = %s" % (n, n, b))
    for k in range(n - 1, -1, -1):
        nuevo = coefs[k] + (x_eval - Xf[k]) * b
        print("  b%d = c%d + (x - x%d)·b%d = %s + (%s)·(%s) = %s"
              % (k, k, k, k + 1, coefs[k], x_eval - Xf[k], b, nuevo))
        b = nuevo
```

```
print()
print("=" * 74)
print("RESULTADO")
print("=" * 74)
p = evaluar_newton(coefs, Xf, x_eval)
print("  P%d(%g) = %.4f ms" % (n, x_eval, float(p)))
print()
print("  Grado por grado (cada uno agrega un término al anterior):")
for g in range(1, n + 1):
    print("    P%d(%g) = %9.4f ms   con los nodos x = %s"
          % (g, x_eval, float(evaluar_newton(coefs[:g + 1], Xf, x_eval)), X[:g + 1]))
print()
print("  Verificación en los nodos:")
for a, c in zip(X, Y):
    v = float(evaluar_newton(coefs, Xf, a))
    print("    P%d(%2g) = %8.4f   (dato: %g)   error = %.2e" % (n, a, v, c, abs(v - c)))

# ---------------------------------------------------------------------
# Gráfica: los 4 puntos, las curvas de grado 1, 2 y 3 y el punto x = 5
# ---------------------------------------------------------------------
try:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    cf = [float(c) for c in coefs]
    malla = [X[0] + i * (X[-1] - X[0]) / 400 for i in range(401)]
    pv = float(p)

    fig, ax = plt.subplots(figsize=(7.4, 4.3), dpi=200)
    ax.plot(malla, [evaluar_newton(cf[:2], X, t) for t in malla], color="#8a97ad", linewidth=1.1,
            linestyle="--", label="P₁(x) · 2 puntos", zorder=1)
    ax.plot(malla, [evaluar_newton(cf[:3], X, t) for t in malla], color="#5b7fb8", linewidth=1.2,
            linestyle="-.", label="P₂(x) · 3 puntos", zorder=1)
    curva = [evaluar_newton(cf, X, t) for t in malla]
    ax.plot(malla, curva, color="#14366b", linewidth=2, label="P₃(x) interpolante", zorder=2)
    ax.scatter(X, Y, s=48, color="#101c33", zorder=3, label="puntos medidos")
    ax.scatter([x_eval], [pv], s=90, color="#c8890a", zorder=4,
               edgecolor="white", linewidth=1.4, label="P₃(5) = %.2f ms" % pv)
    ax.vlines(x_eval, min(curva) - 12, pv, color="#c8890a", linestyle=":", linewidth=1.2, zorder=1)
    ax.annotate("500 req/s → %.2f ms" % pv, (x_eval, pv), textcoords="offset points",
                xytext=(-104, 12), color="#8a6a12", fontsize=9)
    for a, c in zip(X, Y):
        ax.annotate("(%g, %g)" % (a, c), (a, c), textcoords="offset points",
                    xytext=(8, -13), color="#3c4a63", fontsize=8)
    ax.set_xlabel("Carga concurrente x (× 100 req/s)")
    ax.set_ylabel("Latencia media y (ms)")
    ax.set_title("Interpolación de Newton de la latencia frente a la carga")
    ax.grid(color="#dfe5ef", linewidth=0.8)
    ax.set_axisbelow(True)
    for lado in ("top", "right"):
        ax.spines[lado].set_visible(False)
    ax.legend(frameon=False, fontsize=9)
    fig.tight_layout()
    fig.savefig("interpolacion_s9.png")
    print()
    print("  Gráfica guardada en interpolacion_s9.png")
except ImportError:
    print()
    print("  (matplotlib no está instalado: se omitió la gráfica)")
```

## Salida del programa

```
================================================================================
DATOS EXPERIMENTALES
================================================================================
   i   x_i (x100 req/s)     y_i (ms)
   0           1.00          45.00
   1           2.00          65.00
   2           4.00         110.00
   3           7.00         220.00

================================================================================
TABLA DE DIFERENCIAS DIVIDIDAS (matriz triangular)
================================================================================
            i        x_i     f[x_i]   1.a dif.   2.a dif.   3.a dif.
            0          1         45         20        5/6        1/3
            1          2         65       45/2       17/6
            2          4        110      110/3
            3          7        220

  En decimales:
            0          1    45.0000    20.0000     0.8333     0.3333
            1          2    65.0000    22.5000     2.8333
            2          4   110.0000    36.6667
            3          7   220.0000

  Coeficientes (primera fila): 45, 20, 5/6, 1/3

================================================================================
POLINOMIO INTERPOLADOR
================================================================================
  Forma de Newton : P3(x) = 45 + 20(x - 1) + (5/6)(x - 1)(x - 2) + (1/3)(x - 1)(x - 2)(x - 4)
  Forma estándar  : P3(x) = 1/3·x^3 - 3/2·x^2 + 133/6·x + 24
  coeficientes decimales: +0.333333, -1.500000, +22.166667, +24.000000

================================================================================
EVALUACIÓN ANIDADA EN x = 5
================================================================================
  b3 = c3 = 1/3
  b2 = c2 + (x - x2)·b3 = 5/6 + (1)·(1/3) = 7/6
  b1 = c1 + (x - x1)·b2 = 20 + (3)·(7/6) = 47/2
  b0 = c0 + (x - x0)·b1 = 45 + (4)·(47/2) = 139

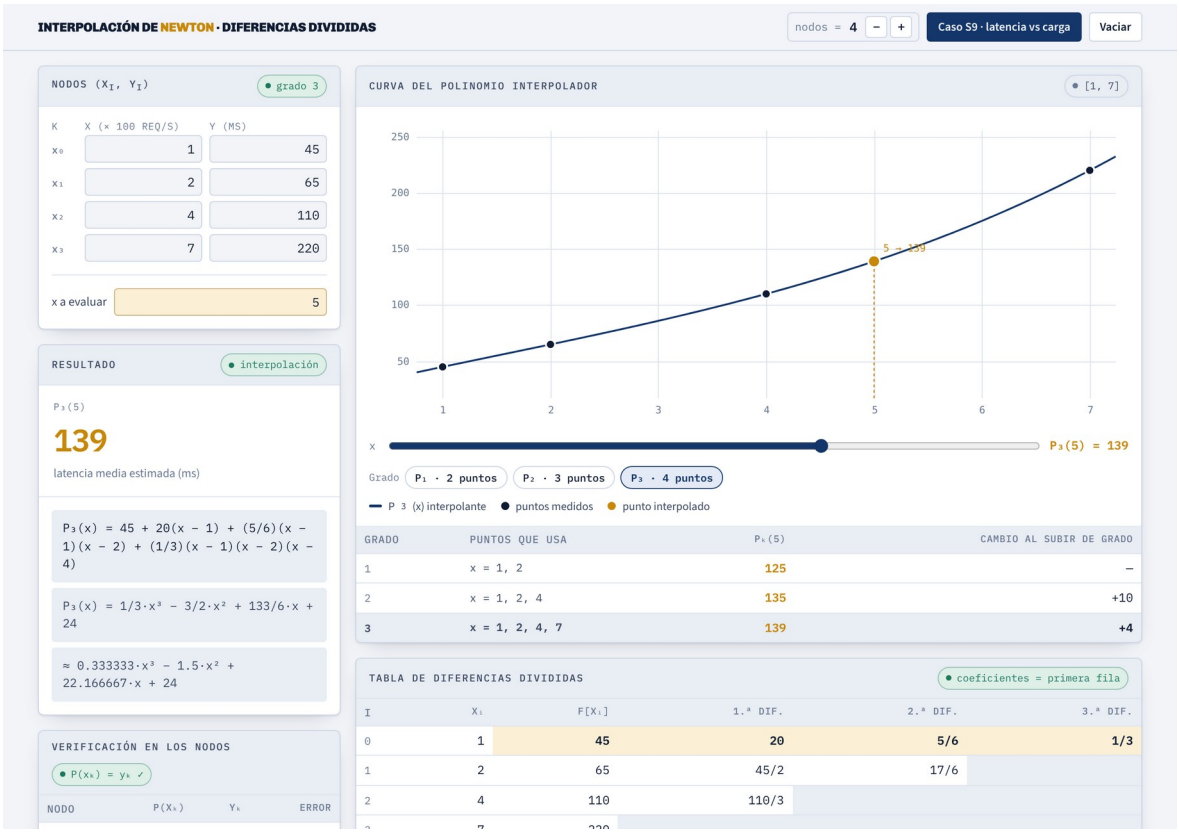
================================================================================
RESULTADO
================================================================================
  P3(5) = 139.0000 ms

  Grado por grado (cada uno agrega un término al anterior):
    P1(5) =  125.0000 ms   con los nodos x = [1, 2]
    P2(5) =  135.0000 ms   con los nodos x = [1, 2, 4]
    P3(5) =  139.0000 ms   con los nodos x = [1, 2, 4, 7]

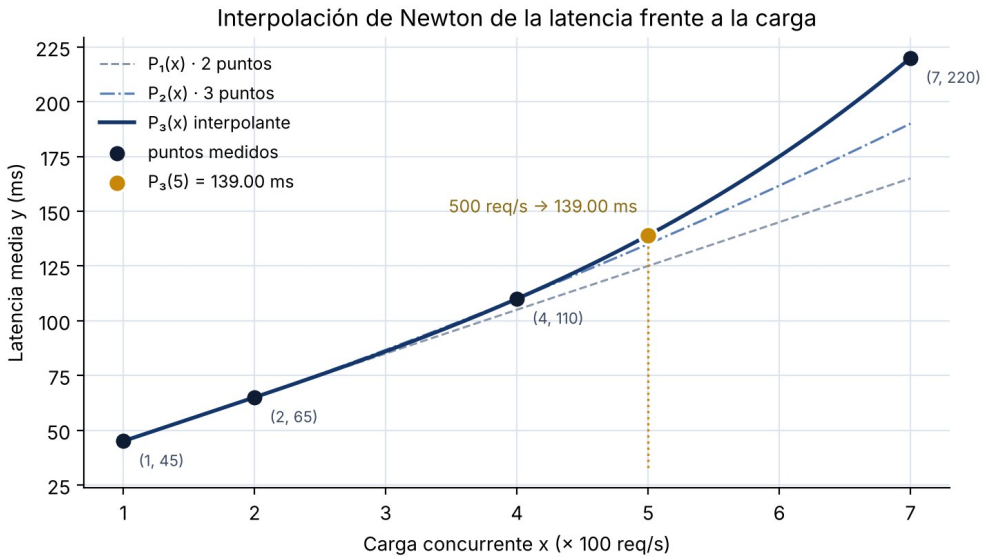
  Verificación en los nodos:
    P3( 1) =  45.0000   (dato: 45)   error = 0.00e+00
    P3( 2) =  65.0000   (dato: 65)   error = 0.00e+00
    P3( 4) = 110.0000   (dato: 110)   error = 0.00e+00
    P3( 7) = 220.0000   (dato: 220)   error = 0.00e+00

  Gráfica guardada en interpolacion_s9.png
```

### Interfaz gráfica




*Aplicativo web: nodos, valor interpolado, curva de $P_3(x)$ con el punto (5; 139) resaltado, deslizador para recorrer la curva y comparación de grados.*




*Gráfica generada por el programa: puntos experimentales, polinomios de grado 1, 2 y 3 y el punto interpolado (5; 139).*

## 8. Análisis de resultados

**Lectura del resultado.** A 500 req/s la latencia estimada es de 139 ms, frente a los 110 ms medidos a 400 req/s: 100 req/s adicionales cuestan unos 29 ms. El valor queda entre las dos mediciones que lo rodean (110 y 220 ms), como corresponde a una interpolación.

**Comportamiento del modelo.** Las diferencias divididas de primer orden crecen de 20 a 22.5 y luego a 36.67 ms por cada 100 req/s, y las de segundo y tercer orden son positivas. La latencia no aumenta en línea recta: el servicio se degrada cada vez más rápido conforme sube la carga, que es el comportamiento esperado cuando una API se acerca a su punto de saturación.

**Limitaciones.** El punto x = 5 cae dentro del rango medido [1, 7], de modo que se trata de una interpolación y no de una extrapolación. Fuera de ese rango el polinomio ya no está sujeto por ningún dato y el error puede crecer sin control. Además, la función solo se conoce por una tabla de mediciones, por lo que no es posible calcular la cota teórica del error de truncamiento, que exige la derivada de cuarto orden de f(x). Con cuatro puntos, la estimación debería contrastarse con una prueba de carga real a 500 req/s antes de usarla para dimensionar el servicio.

## 9. Conclusiones

Se construyó la tabla de diferencias divididas para los cuatro puntos medidos y se obtuvo el polinomio interpolador $P_3(x) = 45 + 20(x - 1) + (5/6)(x - 1)(x - 2) + (1/3)(x - 1)(x - 2)(x - 4)$, cuya forma estándar es $P_3(x) = (1/3)x^3 - (3/2)x^2 + (133/6)x + 24$ y que reproduce exactamente los valores experimentales. Su evaluación en x = 5 arroja una latencia media estimada de 139 ms, resultado confirmado tanto por el desarrollo manual como por el programa en Python y el aplicativo web. Frente a Lagrange, el método de Newton permitió comparar los polinomios de grado 1, 2 y 3 agregando un término cada vez (125, 135 y 139 ms), sin rehacer el cálculo.

---